From assembly to bytes and back

Java implementation | Existing FizzBuzz program | October 25, 2026

A file-based assembler and disassembler for the MIPS instructions used by the supplied FizzBuzz program. The submission includes manual encodings, both machine-code representations, complete source code, and reproducible test evidence.

Public GitHub project

https://github.com/tenth-package0/mips-assembler-disassembler

Only this assignment project is in the repository. Download the project files from GitHub, or extract the embedded files from this PDF using a PDF viewer with an Attachments panel.

Verified results

Check	Result
Encoding and validation tests	29 passed
Instructions / data	45 instructions / 10 data bytes
ASCII and binary round trips	Both files identical after reassembly
Independent MARS execution	Original + 2 reconstructions: 100 lines each

What is included

Parts 1 and 2 show instruction fields, binary bytes, and hexadecimal values. The README documents file inputs, commands, addresses, byte order, and limitations. Listings include the original assembly, assembler, image format helper, disassembler, reconstructed assembly, machine-code files, test code, and captured results.

Binary inclusion: fizzbuzz.mhc-bin is embedded as an actual 210-byte file. Its printed hexadecimal listing is also included for inspection. Every project file other than this PDF is attached with its original bytes.

Provenance: the original FizzBuzz statements and assembler core came from the user-supplied Java file. OpenAI Codex assisted with the extensions, verification, and report. Detailed acknowledgments appear below and in README.md.

Part 1 / Assemble by hand

32-bit MIPS encodings, with each field shown explicitly

Registers: $zero = 0 (00000), $t0 = 8 (01000), $t1 = 9 (01001), $t2 = 10 (01010), and $t3 = 11 (01011).

I format: opcode[6] | rs[5] | rt[5] | immediate[16]. For addi, rs is the source and rt is the destination. Opcode 8 is 001000; positive immediates 3 and 5 are zero-extended to their 16-bit representation.

addi $t0, $zero, 3

```
opcode | rs    | rt    | immediate
001000 | 00000 | 01000 | 0000000000000011
32 bits: 00100000000010000000000000000011
```

addi $t1, $zero, 5

```
opcode | rs    | rt    | immediate
001000 | 00000 | 01001 | 0000000000000101
32 bits: 00100000000010010000000000000101
```

R format: opcode[6] | rs[5] | rt[5] | rd[5] | shamt[5] | funct[6]. Opcode is zero; rs and rt are sources and rd is the destination. Both instructions use shamt = 0. add uses funct = 32 (100000); sub uses funct = 34 (100010).

add $t2, $t0, $t1

```
opcode | rs    | rt    | rd    | shamt | funct
000000 | 01000 | 01001 | 01010 | 00000 | 100000
32 bits: 00000001000010010101000000100000
```

sub $t3, $t1, $t0

```
opcode | rs    | rt    | rd    | shamt | funct
000000 | 01001 | 01000 | 01011 | 00000 | 100010
32 bits: 00000001001010000101100000100010
```

For sub, the operand order matters: $t3 receives $t1 - $t0. With the initial values above, add produces 8 and sub produces 2.

Part 2 / Digits versus raw bytes

Big-endian: the most significant byte is stored first

Instruction	Four 8-bit bytes	Hex bytes
`addi $t0, $zero, 3`	`00100000 00001000 00000000 00000011`	`20 08 00 03`
`addi $t1, $zero, 5`	`00100000 00001001 00000000 00000101`	`20 09 00 05`
`add $t2, $t0, $t1`	`00000001 00001001 01010000 00100000`	`01 09 50 20`
`sub $t3, $t1, $t0`	`00000001 00101000 01011000 00100010`	`01 28 58 22`

mhc-ascii

The file contains readable ASCII characters. Each printed 0 is stored as byte 0x30, and each printed 1 is stored as byte 0x31. A 32-bit instruction needs 32 text characters, which occupy 32 bytes, excluding spaces and line breaks.

mhc-bin

The file contains actual machine-code bytes. The first instruction is stored as four bytes with hexadecimal values 20, 08, 00, and 03. It does not contain the text "20 08 00 03". A text editor may show control characters or unreadable symbols.

For the four Part 1 instructions, the payload needs 128 ASCII digit bytes or 16 raw binary bytes. Spaces, line breaks, and a container header are additional overhead. Big-endian order is a documented choice for this project, not a claim that every MIPS environment uses that order.

How this project stores both formats

Both outputs include a 20-byte MHC1 header before the instructions and data. The ASCII file renders every container byte as eight 0/1 digits. The binary file stores the same bytes directly. The header records segment addresses and lengths, so the disassembler never guesses which bytes are instructions.

FizzBuzz has 45 instructions (180 bytes), followed by 10 bytes containing Fizz, a zero terminator, Buzz, and another zero terminator. With the header, the raw file is 210 bytes. Its ASCII representation contains 1,680 binary digits plus 210 whitespace bytes, for 1,890 bytes total.

README / Run the project

File input, output files, and the round-trip verification commands

Requirements: JDK 8 or later. Run the following commands from the downloaded project directory. No Java libraries are needed. The optional verification script requires Python 3.8 or later.

```
javac Assembler.java Image.java Disassembler.java Tests.java
java Assembler fizzbuzz.asm
java Disassembler fizzbuzz.mhc-bin reconstructed.asm
java Assembler reconstructed.asm roundtrip
java Disassembler fizzbuzz.mhc-ascii reconstructed-ascii.asm
java Assembler reconstructed-ascii.asm roundtrip-ascii
java Tests
python3 verify.py
```

Assembler accepts a source filename and an optional output prefix. With fizzbuzz.asm it writes fizzbuzz.mhc-ascii and fizzbuzz.mhc-bin. Disassembler accepts either image extension and an output assembly filename. Existing output files are replaced; the output directory must already exist.

Verify actual FizzBuzz behavior in MARS

Download Mars4_5.jar from the official release linked in the sources. Use default memory settings and delayed branching OFF. The CLI commands omit db, which leaves delayed branching disabled. The program starts at main, the first text instruction.

```
python3 verify.py --mars /path/to/Mars4_5.jar
java -jar /path/to/Mars4_5.jar nc sm fizzbuzz.asm
java -jar /path/to/Mars4_5.jar nc sm reconstructed.asm
```

With --mars, verify.py checks the original and both reconstructed programs against an independently generated 100-line expected result. Without --mars, it checks encodings and both file round trips and explicitly reports that execution testing was skipped.

Useful error messages

Both Java tools report input and I/O failures to stderr and exit with code 1. Assembly errors include a source line number where applicable. For example, mul $t0, $t1, $t2 on line 2 produces:

```
Assembly failed: Line 2: Unsupported instruction: mul
```

The .mhc format is a custom container, not an ELF executable or directly loadable MARS image. Execute the reconstructed assembly in MARS.

README / Supported subset and layout

All information needed to reproduce and decode this project

Real instructions: addi, addiu, add, addu, sub, ori, lui, div (two registers), mfhi, sll, beq, bne, j, jal, jr, lw, sw, syscall. **Pseudos:** li, la, move, rem with an immediate divisor, nop. **Directives:** .text, .data, .globl, .global, .asciiz, .byte.

Pass 1 expands pseudoinstructions and records label addresses. Pass 2 encodes the expanded instructions. Branches use (target - (PC + 4)) / 4; jumps encode target bits 27..2. Signed immediates and branch distances are checked for range.

li uses addiu, ori, or lui/ori according to the value. la always uses lui/ori. move uses addu, nop uses sll, and rem uses a load into $at followed by div and mfhi. rem rejects zero divisors and operands using $at; it omits MARS runtime division checks. FizzBuzz uses positive values and divisors 3 and 5.

Offset	Bytes	MHC1 field (big-endian)
0	4	Magic/version: 4D 48 43 31 (MHC1)
4	4	Text base: 0x00400000
8	4	Text length: 180 bytes
12	4	Data base: 0x10010000
16	4	Data length: 10 bytes
20	180	Instruction payload
200	10	Data payload; no padding

Only the fixed bases above are supported. The address gap is not stored. Data is packed without implicit alignment. Entry is the first text instruction. Labels are case sensitive; named and numeric registers and decimal/hex/binary literals are accepted. Strings and character literals are ASCII. Unlisted instructions, directives, and arithmetic expressions are unsupported.

The disassembler emits numeric registers, generated address labels, real instructions, and .byte data. Comments, original names, and pseudo spellings are not recovered. It reassembles the result and rejects any byte mismatch, including noncanonical reserved bits. This preserves the program and its data exactly.

Verification and acknowledgments

Independent execution plus exact file comparisons

Tests were run with OpenJDK 17 and MARS 4.5. All 29 encoding, parsing, format-validation, and rejection tests passed. Tests cover the four manual words, branch offsets, jump targets, pseudo expansion, every supported real instruction, data preservation, invalid operands, malformed headers, and invalid ASCII input.

The original and both reconstructed sources each produced the expected 100 FizzBuzz lines in MARS. The reconstructed sources were identical. Reassembly reproduced both the raw binary image and its ASCII representation byte for byte. Complete captured evidence follows the source listings.

```
Binary image: 210 bytes
SHA-256:
521b0b9dbde491f888cdd0c7047bc5a382af40e213d60275bba391153022553f
```

Sources and collaboration

The user supplied Assembler (14) (1).java. Its parser, encoder core, and existing FizzBuzz DEMO form the starting point. The FizzBuzz statements were extracted into fizzbuzz.asm; its return-address save/restore and control flow were retained. OpenAI Codex assisted with file I/O, the MHC1 format, disassembly, additional supported operations, tests, documentation, and this PDF.

Assigned reading: ZyBooks Chapter 4, as stated in the assignment. The chapter text was not provided or consulted for this submission.

MIPS field layouts and opcode reference (University of Waterloo)
https://student.cs.uwaterloo.ca/~isg/res/mips/opcodes

MARS settings / delayed branching
https://dpetersanderson.github.io/Help/MarsHelpSettings.html

MARS syscall services used: 1, 4, 10, and 11
https://dpetersanderson.github.io/Help/SyscallHelp.html

MARS command-line options
https://dpetersanderson.github.io/Help/MarsHelpCommand.html

Official MARS release used for testing
https://github.com/dpetersanderson/MARS/releases/tag/v.4.5.1

Original FizzBuzz source

fizzbuzz.asm | listing 1 of 1

```
# Extracted from the supplied Assembler Java DEMO.
# MARS: delayed branching disabled; prints 1 through 100.
.data
fizz: .asciiz "Fizz"
buzz: .asciiz "Buzz"
.text
.globl main
main:
    li $t1, 1
    li $t2, 101
Loop:
    jal PrintItem
    jal PrintNewLine
    addi $t1, $t1, 1
    bne $t1, $t2, Loop
    j Exit
PrintItem:
    addiu $sp, $sp, -8
    sw $ra, 4($sp)
    rem $t3, $t1, 3
    rem $t4, $t1, 5
    bne $t3, $zero, CheckBuzz
    jal PrintFizz
CheckBuzz:
    bne $t4, $zero, CheckNumber
    jal PrintBuzz
CheckNumber:
    beq $t3, $zero, PrintItemDone
    beq $t4, $zero, PrintItemDone
    jal PrintInt
PrintItemDone:
    lw $ra, 4($sp)
    addiu $sp, $sp, 8
    jr $ra
PrintFizz:
    li $v0, 4
    la $a0, fizz
    syscall
    jr $ra
PrintBuzz:
    li $v0, 4
    la $a0, buzz
    syscall
    jr $ra
PrintInt:
    li $v0, 1
    move $a0, $t1
    syscall
    jr $ra
PrintNewLine:
    li $v0, 11
    li $a0, '\n'
    syscall
    jr $ra
Exit:
    li $v0, 10
    syscall
```

Assembler source

Assembler.java | listing 1 of 9

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/** Two-pass, file-based MIPS subset assembler. Java 8+. */
public class Assembler {
    static final int TEXT_BASE = 0x00400000;
    static final int DATA_BASE = 0x10010000;

    private static final String[] REGISTER_NAMES = {
        "zero", "at", "v0", "v1", "a0", "a1", "a2", "a3",
        "t0", "t1", "t2", "t3", "t4", "t5", "t6", "t7",
        "s0", "s1", "s2", "s3", "s4", "s5", "s6", "s7",
        "t8", "t9", "k0", "k1", "gp", "sp", "fp", "ra"
    };

    static class Instruction {
        final int line, address;
        final String text, sourceText;
        Instruction(int line, int address, String text, String sourceText) {
            this.line = line;
            this.address = address;
            this.text = text;
            this.sourceText = sourceText;
        }
    }

    static class EncodedInstruction {
        final Instruction source;
        final int word;
        final String mips;
        final Map<String, Integer> labelLines;
        EncodedInstruction(Instruction source, int word, String mips,
                           Map<String, Integer> labelLines) {
            this.source = source;
            this.word = word;
            this.mips = mips;
            this.labelLines = labelLines;
        }
    }

    static class Result {
        final byte[] text, data;
        final List<EncodedInstruction> instructions;
        Result(byte[] text, byte[] data, List<EncodedInstruction> instructions) {
            this.text = text;
            this.data = data;
            this.instructions = instructions;
        }
    }

    private final Map<String, Integer> labels = new HashMap<>();
    private final Map<String, Integer> labelLines = new HashMap<>();
    private final Set<String> textLabels = new HashSet<>();
    private final List<String> globals = new ArrayList<>();
```

Assembler source / continued

Assembler.java | listing 2 of 9

```
    private final List<Instruction> instructions = new ArrayList<>();
    private final List<Byte> data = new ArrayList<>();
    private boolean inData = false;
    private int pc = TEXT_BASE;

    public static void main(String[] args) {
        try {
            if (args.length < 1 || args.length > 2) {
                fail("Usage: java Assembler <source.asm> [output-prefix]");
            }
            java.nio.file.Path input = java.nio.file.Paths.get(args[0]);
            String source = new String(java.nio.file.Files.readAllBytes(input),
                                       java.nio.charset.StandardCharsets.UTF_8);
            String prefix = args.length == 2 ? args[1]
                : args[0].replaceFirst("\\.[^./\\\\]+$", "");
            Result result = assemble(source);
            Image.write(prefix, result.text, result.data);
            System.out.println("Assembled " + result.instructions.size()
                + " instructions; " + result.data.length + " data bytes.");
            System.out.println("Wrote " + prefix + ".mhc-ascii and "
                + prefix + ".mhc-bin");
        } catch (java.io.IOException | IllegalArgumentException e) {
            System.err.println("Assembly failed: " + e.getMessage());
            System.exit(1);
        }
    }

    static Result assemble(String source) {
        Assembler assembler = new Assembler();
        assembler.firstPass(source);
        if (assembler.instructions.isEmpty()) fail("No instructions found.");
        for (String name : assembler.globals) assembler.label(name);

        // Pass 2: all labels now have final addresses.
        byte[] text = new byte[assembler.instructions.size() * 4];
        List<EncodedInstruction> encoded = new ArrayList<>();
        int index = 0;
        for (Instruction instruction : assembler.instructions) {
            try {
                int word = assembler.encode(instruction);
                encoded.add(new EncodedInstruction(instruction, word,
                            assembler.resolvedMips(instruction), assembler.labelLines));
                for (int shift = 24; shift >= 0; shift -= 8) {
                    text[index++] = (byte) (word >>> shift);
                }
            } catch (IllegalArgumentException e) {
                throw new IllegalArgumentException("Line " + instruction.line + ": " + e.getMessage());
            }
        }
        byte[] bytes = new byte[assembler.data.size()];
        for (int i = 0; i < bytes.length; i++) bytes[i] = assembler.data.get(i);
        return new Result(text, bytes, encoded);
    }

    // Pass 1: expand pseudos BEFORE calculating following label addresses.
    private void firstPass(String source) {
        if (source.startsWith("\uFEFF")) source = source.substring(1);
        String[] lines = source.split("\\r\\n|\\r|\\n");
```

Assembler source / continued

Assembler.java | listing 3 of 9

```
        for (int i = 0; i < lines.length; i++) {
            try {
                String text = withoutComment(lines[i]).trim();
                while (true) {
                    int colon = text.indexOf(':');
                    if (colon < 0) break;
                    String name = text.substring(0, colon).trim();
                    if (!name.matches("[A-Za-z_][A-Za-z0-9_]*")) break;
                    if (labels.containsKey(name)) fail("Duplicate label: " + name);
                    labels.put(name, inData ? DATA_BASE + data.size() : pc);
                    labelLines.put(name, i + 1);
                    if (!inData) textLabels.add(name);
                    text = text.substring(colon + 1).trim();
                }
                if (text.isEmpty()) continue;
                if (text.startsWith(".")) {
                    directive(text);
                } else {
                    if (inData) fail("Instructions must be in .text.");
                    expand(text, i + 1);
                }
            } catch (IllegalArgumentException e) {
                throw new IllegalArgumentException("Line " + (i + 1) + ": " + e.getMessage());
            }
        }
    }

    private void directive(String text) {
        String[] p = text.split("\\s+", 2);
        String value = p.length == 2 ? p[1].trim() : "";
        switch (p[0]) {
            case ".text":
            case ".data":
                boolean nextData = p[0].equals(".data");
                int base = nextData ? DATA_BASE : TEXT_BASE;
                if (!value.isEmpty()) {
                    if (number(value) != base) fail("Only the default segment base is supported.");
                    if (nextData ? !data.isEmpty() : !instructions.isEmpty()) {
                        fail("Explicit segment address must precede its contents.");
                    }
                }
                inData = nextData;
                break;
            case ".globl":
            case ".global":
                if (!value.matches("[A-Za-z_][A-Za-z0-9_]*")) fail("Expected one global label.");
                globals.add(value);
                break;
            case ".byte":
                if (!inData) fail(".byte must be in .data.");
                for (String item : value.split(",", -1)) {
                    data.add((byte) range(number(item), 0, 255));
                }
                break;
            case ".asciiz":
                if (!inData) fail(".asciiz must be in .data.");
                String decoded = quoted(value, '"');
                for (int i = 0; i < decoded.length(); i++) data.add((byte) decoded.charAt(i));
```

Assembler source / continued

Assembler.java | listing 4 of 9

```
                data.add((byte) 0);
                break;
            default:
                fail("Unsupported directive: " + p[0]);
        }
    }

    private void emit(String text, int line, String sourceText) {
        instructions.add(new Instruction(line, pc, text, sourceText));
        pc += 4;
    }

    private void loadImmediate(String register, int value, int line, String sourceText) {
        reg(register);
        if (value >= -32768 && value <= 32767) {
            emit("addiu " + register + ", $zero, " + value, line, sourceText);
        } else if (value >= 0 && value <= 65535) {
            emit("ori " + register + ", $zero, " + value, line, sourceText);
        } else {
            emit("lui " + register + ", " + (value >>> 16), line, sourceText);
            emit("ori " + register + ", " + register + ", " + (value & 65535), line, sourceText);
        }
    }

    private void expand(String text, int line) {
        String[] p = text.split("\\s+", 2);
        String op = lowerAscii(p[0]);
        String[] a = operands(p);
        switch (op) {
            case "li":
                count(a, 2);
                loadImmediate(a[0], number(a[1]), line, text);
                break;
            case "la":
                count(a, 2);
                reg(a[0]);
                if (!a[1].matches("[A-Za-z_][A-Za-z0-9_]*")) fail("la expects a label.");
                emit("lui " + a[0] + ", hi(" + a[1] + ")", line, text);
                emit("ori " + a[0] + ", " + a[0] + ", lo(" + a[1] + ")", line, text);
                break;
            case "move":
                count(a, 2);
                emit("addu " + a[0] + ", " + a[1] + ", $zero", line, text);
                break;
            case "rem":
                count(a, 3);
                if (reg(a[0]) == 1 || reg(a[1]) == 1) fail("rem reserves $at.");
                int divisor = number(a[2]);
                if (divisor == 0) fail("rem divisor cannot be zero.");
                loadImmediate("$at", divisor, line, text);
                emit("div " + a[1] + ", $at", line, text);
                emit("mfhi " + a[0], line, text);
                break;
            case "nop":
                count(a, 0);
                emit("sll $zero, $zero, 0", line, text);
                break;
            default:
```

Assembler source / continued

Assembler.java | listing 5 of 9

```
                emit(text, line, text);
            }
        }

        private int encode(Instruction instruction) {
            String[] p = instruction.text.split("\\s+", 2);
            String op = lowerAscii(p[0]);
            String[] a = operands(p);
            switch (op) {
                case "addi":
                case "addiu":
                case "ori":
                    count(a, 3);
                    int immediate = value(a[2]);
                    range(immediate, op.equals("ori") ? 0 : -32768,
                                     op.equals("ori") ? 65535 : 32767);
                    return iType(op.equals("addi") ? 8 : op.equals("addiu") ? 9 : 13,
                                 reg(a[1]), reg(a[0]), immediate);
                case "lui":
                    count(a, 2);
                    return iType(15, 0, reg(a[0]), range(value(a[1]), 0, 65535));
                case "add":
                case "sub":
                case "addu":
                    count(a, 3);
                    int function = op.equals("add") ? 0x20
                        : op.equals("sub") ? 0x22 : 0x21;
                    return rType(reg(a[1]), reg(a[2]), reg(a[0]), 0, function);
                case "div":
                    count(a, 2);
                    return rType(reg(a[0]), reg(a[1]), 0, 0, 0x1A);
                case "mfhi":
                    count(a, 1);
                    return rType(0, 0, reg(a[0]), 0, 0x10);
                case "sll":
                    count(a, 3);
                    return rType(0, reg(a[1]), reg(a[0]), range(number(a[2]), 0, 31), 0);
                case "beq":
                case "bne":
                    count(a, 3);
                    int offset = (codeLabel(a[2]) - (instruction.address + 4)) / 4;
                    return iType(op.equals("beq") ? 4 : 5, reg(a[0]), reg(a[1]),
                                 range(offset, -32768, 32767));
                case "j":
                case "jal":
                    count(a, 1);
                    int target = codeLabel(a[0]);
                    if ((target & 0xF0000000) != ((instruction.address + 4) & 0xF0000000)) {
                        fail("Jump target is out of range.");
                    }
                    return ((op.equals("j") ? 2 : 3) << 26) | ((target >>> 2) & 0x03FFFFFF);
                case "jr":
                    count(a, 1);
                    return rType(reg(a[0]), 0, 0, 0, 8);
                case "lw":
                case "sw":
                    count(a, 2);
                    String memory = a[1].replaceAll("\\s+", "");
```

Assembler source / continued

Assembler.java | listing 6 of 9

```
                if (!memory.matches("[^()]+\\(\\$[A-Za-z0-9]+\\)")) {
                    fail("Expected offset($register): " + a[1]);
                }
                int left = memory.indexOf('(');
                int base = reg(memory.substring(left + 1, memory.length() - 1));
                int displacement = range(number(memory.substring(0, left)), -32768, 32767);
                return iType(op.equals("lw") ? 35 : 43, base, reg(a[0]), displacement);
            case "syscall":
                count(a, 0);
                return 0x0000000C;
            default:
                throw new IllegalArgumentException("Unsupported instruction: " + op);
        }
    }

    // R: opcode(6), rs(5), rt(5), rd(5), shamt(5), funct(6).
    private static int rType(int rs, int rt, int rd, int shift, int function) {
        return (rs << 21) | (rt << 16) | (rd << 11) | (shift << 6) | function;
    }

    // I: opcode(6), rs(5), rt(5), immediate(16).
    private static int iType(int opcode, int rs, int rt, int immediate) {
        return (opcode << 26) | (rs << 21) | (rt << 16) | (immediate & 0xFFFF);
    }

    private int label(String name) {
        Integer result = labels.get(name);
        if (result == null) fail("Undefined label: " + name);
        return result;
    }

    private int codeLabel(String name) {
        int address = label(name);
        if (!textLabels.contains(name)) fail("Branch/jump target is not in .text: " + name);
        return address;
    }

    private int value(String text) {
        if (text.startsWith("hi(") && text.endsWith(")")) {
            return label(text.substring(3, text.length() - 1)) >>> 16;
        }
        if (text.startsWith("lo(") && text.endsWith(")")) {
            return label(text.substring(3, text.length() - 1)) & 65535;
        }
        return number(text);
    }

    private static int reg(String name) {
        name = lowerAscii(name.trim());
        if (name.equals("$s8")) return 30;
        for (int i = 0; i < REGISTER_NAMES.length; i++) {
            if (name.equals("$" + REGISTER_NAMES[i]) || name.equals("$" + i)) return i;
        }
        throw new IllegalArgumentException("Unknown register: " + name);
    }

    private static int number(String text) {
        text = text.trim();
```

Assembler source / continued

Assembler.java | listing 7 of 9

```
        if (text.startsWith("'")) {
            String character = quoted(text, '\'');
            if (character.length() != 1) fail("Expected one character: " + text);
            return character.charAt(0);
        }
        try {
            boolean negative = text.startsWith("-");
            String digits = text;
            if (digits.startsWith("-") || digits.startsWith("+")) digits = digits.substring(1);
            int radix = 10;
            if (digits.startsWith("0x") || digits.startsWith("0X")) {
                radix = 16;
                digits = digits.substring(2);
            } else if (digits.startsWith("0b") || digits.startsWith("0B")) {
                radix = 2;
                digits = digits.substring(2);
            }
            if (digits.isEmpty() || digits.startsWith("-") || digits.startsWith("+")) {
                fail("Invalid integer: " + text);
            }
            long result = Long.parseLong(digits, radix);
            if (negative) result = -result;
            if (result < Integer.MIN_VALUE || result > 0xFFFFFFFFL) fail("Integer too large: " + text);
            return (int) result;
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Invalid integer: " + text);
        }
    }

    // Keep # inside quoted strings or character literals.
    private static String withoutComment(String text) {
        char quote = 0;
        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);
            if (quote != 0) {
                if (c == '\\') i++;
                else if (c == quote) quote = 0;
            } else if (c == '"' || c == '\'') {
                quote = c;
            } else if (c == '#') {
                return text.substring(0, i);
            }
        }
        if (quote != 0) fail("Unclosed quote.");
        return text;
    }

    private static String quoted(String text, char quote) {
        if (text.length() < 2 || text.charAt(0) != quote
                || text.charAt(text.length() - 1) != quote) fail("Expected quoted literal: " + text);
        StringBuilder decoded = new StringBuilder();
        for (int i = 1; i < text.length() - 1; i++) {
            char c = text.charAt(i);
            if (c == quote) fail("Unexpected quote: " + text);
            if (c == '\\') {
                if (++i >= text.length() - 1) fail("Incomplete escape.");
                switch (text.charAt(i)) {
                    case 'n': c = '\n'; break;
```

Assembler source / continued

Assembler.java | listing 8 of 9

```
                    case 'r': c = '\r'; break;
                    case 't': c = '\t'; break;
                    case '0': c = 0; break;
                    case '\\': c = '\\'; break;
                    case '\'': c = '\''; break;
                    case '"': c = '"'; break;
                    default: fail("Unsupported escape: \\" + text.charAt(i));
                }
            }
            if (c > 127) fail("This toy assembler accepts ASCII literals only.");
            decoded.append(c);
        }
        return decoded.toString();
    }

    private static String[] operands(String[] p) {
        if (p.length == 1 || p[1].trim().isEmpty()) return new String[0];
        // Split commas outside quotes so li $a0, ',' also works.
        List<String> result = new ArrayList<>();
        char quote = 0;
        int start = 0;
        for (int i = 0; i < p[1].length(); i++) {
            char c = p[1].charAt(i);
            if (quote != 0) {
                if (c == '\\') i++;
                else if (c == quote) quote = 0;
            } else if (c == '\'' || c == '"') {
                quote = c;
            } else if (c == ',') {
                result.add(p[1].substring(start, i).trim());
                start = i + 1;
            }
        }
        result.add(p[1].substring(start).trim());
        return result.toArray(new String[0]);
    }

    private static void count(String[] a, int expected) {
        if (a.length != expected) fail("Expected " + expected + " comma-separated operand(s).");
        for (String operand : a) if (operand.isEmpty()) fail("Empty operand.");
    }

    private static int range(int value, int min, int max) {
        if (value < min || value > max) fail("Value out of range: " + value);
        return value;
    }

    private static void fail(String message) {
        throw new IllegalArgumentException(message);
    }

    // Replace assembler-internal hi/lo expressions with the encoded immediates.
    private String resolvedMips(Instruction instruction) {
        String[] p = instruction.text.split("\\s+", 2);
        String op = lowerAscii(p[0]);
        String[] a = operands(p);
        if (op.equals("lui")) return "lui " + a[0] + ", " + value(a[1]);
        if (op.equals("ori")) return "ori " + a[0] + ", " + a[1] + ", " + value(a[2]);
```

Assembler source / continued

Assembler.java | listing 9 of 9

```
465          return instruction.text;
466      }
467
468      private static String lowerAscii(String text) {
469          StringBuilder result = new StringBuilder();
470          for (int i = 0; i < text.length(); i++) {
471              char c = text.charAt(i);
472              result.append(c >= 'A' && c <= 'Z' ? (char) (c + 32) : c);
473          }
474          return result.toString();
475      }
476  }
```

Image format helper

Image.java | listing 1 of 2

```
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Arrays;

/** MHC1 container shared by both tools. All integers are big-endian. */
public final class Image {
    static final int MAGIC = 0x4D484331; // ASCII MHC1
    static final int HEADER_SIZE = 20;
    final byte[] text, data;

    Image(byte[] text, byte[] data) {
        this.text = text;
        this.data = data;
    }

    static byte[] pack(byte[] text, byte[] data) {
        ByteBuffer b = ByteBuffer.allocate(HEADER_SIZE + text.length + data.length)
            .order(ByteOrder.BIG_ENDIAN);
        b.putInt(MAGIC).putInt(Assembler.TEXT_BASE).putInt(text.length);
        b.putInt(Assembler.DATA_BASE).putInt(data.length);
        b.put(text).put(data);
        return b.array();
    }

    static void write(String prefix, byte[] text, byte[] data) throws IOException {
        byte[] raw = pack(text, data);
        StringBuilder ascii = new StringBuilder();
        for (int i = 0; i < raw.length; i++) {
            for (int bit = 7; bit >= 0; bit--) {
                ascii.append(((raw[i] & 255) >>> bit) & 1);
            }
            ascii.append(i % 4 == 3 || i == raw.length - 1 ? '\n' : ' ');
        }
        Files.write(Paths.get(prefix + ".mhc-bin"), raw);
        Files.write(Paths.get(prefix + ".mhc-ascii"),
                    ascii.toString().getBytes(StandardCharsets.US_ASCII));
    }

    static Image read(Path path) throws IOException {
        byte[] raw = Files.readAllBytes(path);
        String name = path.getFileName().toString();
        if (name.endsWith(".mhc-ascii")) {
            StringBuilder bits = new StringBuilder();
            for (byte b : raw) {
                if (b == '0' || b == '1') bits.append((char) b);
                else if (b != ' ' && b != '\n' && b != '\r' && b != '\t') {
                    throw new IllegalArgumentException(
                        "ASCII image accepts only 0, 1, and whitespace.");
                }
            }
            if (bits.length() % 8 != 0) {
                throw new IllegalArgumentException("Incomplete byte in ASCII image.");
            }
```

Image format helper / continued

Image.java | listing 2 of 2

```
            raw = new byte[bits.length() / 8];
            for (int i = 0; i < raw.length; i++) {
                raw[i] = (byte) Integer.parseInt(bits.substring(i * 8, i * 8 + 8), 2);
            }
        } else if (!name.endsWith(".mhc-bin")) {
            throw new IllegalArgumentException("Expected .mhc-ascii or .mhc-bin input.");
        }
        if (raw.length < HEADER_SIZE) {
            throw new IllegalArgumentException("Truncated MHC1 header.");
        }
        ByteBuffer b = ByteBuffer.wrap(raw).order(ByteOrder.BIG_ENDIAN);
        if (b.getInt() != MAGIC) {
            throw new IllegalArgumentException("Bad MHC1 magic/version.");
        }
        int textBase = b.getInt(), textSize = b.getInt();
        int dataBase = b.getInt(), dataSize = b.getInt();
        if (textBase != Assembler.TEXT_BASE || dataBase != Assembler.DATA_BASE) {
            throw new IllegalArgumentException("Unsupported segment starting addresses.");
        }
        if (textSize <= 0 || textSize % 4 != 0 || dataSize < 0
            || HEADER_SIZE + (long) textSize + dataSize != raw.length
            || (long) textBase + textSize > dataBase
            || (long) dataBase + dataSize > Integer.MAX_VALUE) {
            throw new IllegalArgumentException("Invalid segment lengths or file size.");
        }
        return new Image(Arrays.copyOfRange(raw, HEADER_SIZE, HEADER_SIZE + textSize),
            Arrays.copyOfRange(raw, HEADER_SIZE + textSize, raw.length));
    }
}
```

Disassembler source

Disassembler.java | listing 1 of 2

```java
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Arrays;

/** Rebuilds canonical assembly from either MHC1 representation. */
public final class Disassembler {
    public static void main(String[] args) {
        try {
            if (args.length != 2) {
                throw new IllegalArgumentException(
                    "Usage: java Disassembler <input.mhc-bin|input.mhc-ascii> <out.asm>");
            }
            Image image = Image.read(Paths.get(args[0]));
            String source = disassemble(image);
            Files.write(Paths.get(args[1]), source.getBytes(StandardCharsets.UTF_8));
            System.out.println("Wrote " + args[1] + "; reassembly bytes verified.");
        } catch (IOException | IllegalArgumentException e) {
            System.err.println("Disassembly failed: " + e.getMessage());
            System.exit(1);
        }
    }

    static String disassemble(Image image) {
        StringBuilder s = new StringBuilder();
        s.append("# Reconstructed MHC1 image; delayed branching OFF.\n.data\n");
        for (int i = 0; i < image.data.length; i += 16) {
            s.append("    .byte ");
            for (int j = i; j < Math.min(i + 16, image.data.length); j++) {
                if (j != i) s.append(", ");
                s.append(image.data[j] & 255);
            }
            s.append('\n');
        }
        s.append(".text\n.globl main\nmain:\n");
        ByteBuffer b = ByteBuffer.wrap(image.text).order(ByteOrder.BIG_ENDIAN);
        for (int offset = 0; offset < image.text.length; offset += 4) {
            int pc = Assembler.TEXT_BASE + offset;
            int word = b.getInt();
            try {
                s.append(label(pc)).append(":\n    ");
                s.append(decode(word, pc, image.text.length)).append('\n');
            } catch (IllegalArgumentException e) {
                throw new IllegalArgumentException(String.format(
                    "At 0x%08X (word 0x%08X): %s", pc, word, e.getMessage()));
            }
        }
        // A label immediately after the last instruction is a valid source label.
        s.append(label(Assembler.TEXT_BASE + image.text.length)).append(":\n");
        Assembler.Result rebuilt = Assembler.assemble(s.toString());
        if (!Arrays.equals(image.text, rebuilt.text)
            || !Arrays.equals(image.data, rebuilt.data)) {
            throw new IllegalArgumentException(
                "Noncanonical/reserved instruction bits; cannot reproduce bytes.");
        }
```

Disassembler source / continued

Disassembler.java | listing 2 of 2

```
59          return s.toString();
60      }
61
62      private static String reg(int number) { return "$" + number; }
63      private static String label(int address) {
64          return String.format("L_%08X", address);
65      }
66
67      private static String target(int address, int length) {
68          long end = (long) Assembler.TEXT_BASE + length;
69          if (address < Assembler.TEXT_BASE || address > end || (address & 3) != 0) {
70              throw new IllegalArgumentException("Control-flow target outside text segment.");
71          }
72          return label(address);
73      }
74
75      private static String decode(int w, int pc, int length) {
76          int op = w >>> 26, rs = (w >>> 21) & 31, rt = (w >>> 16) & 31;
77          int rd = (w >>> 11) & 31, shift = (w >>> 6) & 31, fn = w & 63;
78          int unsigned = w & 65535, signed = (short) unsigned;
79          if (op == 0) {
80              switch (fn) {
81                  case 0: return "sll " + reg(rd) + ", " + reg(rt) + ", " + shift;
82                  case 8: return "jr " + reg(rs);
83                  case 12: return "syscall";
84                  case 16: return "mfhi " + reg(rd);
85                  case 26: return "div " + reg(rs) + ", " + reg(rt);
86                  case 32: case 33: case 34:
87                      String mnemonic = fn == 32 ? "add" : fn == 33 ? "addu" : "sub";
88                      return mnemonic + " " + reg(rd) + ", " + reg(rs) + ", " + reg(rt);
89                  default: throw new IllegalArgumentException("Unsupported R function: " + fn);
90              }
91          }
92          switch (op) {
93              case 2: case 3:
94                  int address = ((pc + 4) & 0xF0000000) | ((w & 0x03FFFFFF) << 2);
95                  return (op == 2 ? "j " : "jal ") + target(address, length);
96              case 4: case 5:
97                  return (op == 4 ? "beq " : "bne ") + reg(rs) + ", " + reg(rt)
98                      + ", " + target(pc + 4 + signed * 4, length);
99              case 8: case 9: case 13:
100                 String mnemonic = op == 8 ? "addi" : op == 9 ? "addiu" : "ori";
101                 return mnemonic + " " + reg(rt) + ", " + reg(rs) + ", "
102                     + (op == 13 ? unsigned : signed);
103             case 15: return "lui " + reg(rt) + ", " + unsigned;
104             case 35: case 43:
105                 return (op == 35 ? "lw " : "sw ") + reg(rt) + ", "
106                     + signed + "(" + reg(rs) + ")";
107             default: throw new IllegalArgumentException("Unsupported opcode: " + op);
108         }
109     }
110 }
```

Reconstructed assembly

reconstructed.asm | listing 1 of 2

```
# Reconstructed MHC1 image; delayed branching OFF.
.data
    .byte 70, 105, 122, 122, 0, 66, 117, 122, 122, 0
.text
.globl main
main:
L_00400000:
    addiu $9, $0, 1
L_00400004:
    addiu $10, $0, 101
L_00400008:
    jal L_0040001C
L_0040000C:
    jal L_0040009C
L_00400010:
    addi $9, $9, 1
L_00400014:
    bne $9, $10, L_00400008
L_00400018:
    j L_004000AC
L_0040001C:
    addiu $29, $29, -8
L_00400020:
    sw $31, 4($29)
L_00400024:
    addiu $1, $0, 3
L_00400028:
    div $9, $1
L_0040002C:
    mfhi $11
L_00400030:
    addiu $1, $0, 5
L_00400034:
    div $9, $1
L_00400038:
    mfhi $12
L_0040003C:
    bne $11, $0, L_00400044
L_00400040:
    jal L_00400064
L_00400044:
    bne $12, $0, L_0040004C
L_00400048:
    jal L_00400078
L_0040004C:
    beq $11, $0, L_00400058
L_00400050:
    beq $12, $0, L_00400058
L_00400054:
    jal L_0040008C
L_00400058:
    lw $31, 4($29)
L_0040005C:
    addiu $29, $29, 8
L_00400060:
    jr $31
L_00400064:
    addiu $2, $0, 4
```

Reconstructed assembly / continued

reconstructed.asm | listing 2 of 2

```
L_00400068:
    lui $4, 4097
L_0040006C:
    ori $4, $4, 0
L_00400070:
    syscall
L_00400074:
    jr $31
L_00400078:
    addiu $2, $0, 4
L_0040007C:
    lui $4, 4097
L_00400080:
    ori $4, $4, 5
L_00400084:
    syscall
L_00400088:
    jr $31
L_0040008C:
    addiu $2, $0, 1
L_00400090:
    addu $4, $9, $0
L_00400094:
    syscall
L_00400098:
    jr $31
L_0040009C:
    addiu $2, $0, 11
L_004000A0:
    addiu $4, $0, 10
L_004000A4:
    syscall
L_004000A8:
    jr $31
L_004000AC:
    addiu $2, $0, 10
L_004000B0:
    syscall
L_004000B4:
```

ASCII machine-code file

Complete file | first 5 lines are the MHC1 header; line 51 starts data

```
01001101 01001000 01000011 00110001
00000000 01000000 00000000 00000000
00000000 00000000 00000000 10110100
00010000 00000001 00000000 00000000
00000000 00000000 00000000 00001010
00100100 00001001 00000000 00000001
00100100 00001010 00000000 01100101
00001100 00010000 00000000 00000111
00001100 00010000 00000000 00100111
00100001 00101001 00000000 00000001
00010101 00101010 11111111 11111100
00001000 00010000 00000000 00101011
00100111 10111101 11111111 11111000
10101111 10111111 00000000 00000100
00100100 00000001 00000000 00000011
00000001 00100001 00000000 00011010
00000000 00000000 01011000 00010000
00100100 00000001 00000000 00000101
00000001 00100001 00000000 00011010
00000000 00000000 01100000 00010000
00010101 01100000 00000000 00000001
00001100 00010000 00000000 00011001
00010101 10000000 00000000 00000001
00001100 00010000 00000000 00011110
00010001 01100000 00000000 00000010
00010001 10000000 00000000 00000001
00001100 00010000 00000000 00100011
10001111 10111111 00000000 00000100
00100111 10111101 00000000 00001000
00000011 11100000 00000000 00001000
00100100 00000010 00000000 00000100
00111100 00000100 00010000 00000001
00110100 10000100 00000000 00000000
00000000 00000000 00000000 00001100
00000011 11100000 00000000 00001000
00100100 00000010 00000000 00000100
00111100 00000100 00010000 00000001
00110100 10000100 00000000 00000101
00000000 00000000 00000000 00001100
00000011 11100000 00000000 00001000
00100100 00000010 00000000 00000001
00000001 00100000 00100000 00100001
00000000 00000000 00000000 00001100
00000011 11100000 00000000 00001000
00100100 00000010 00000000 00001011
00100100 00000100 00000000 00001010
00000000 00000000 00000000 00001100
00000011 11100000 00000000 00001000
00100100 00000010 00000000 00001010
00000000 00000000 00000000 00001100
01000110 01101001 01111010 01111010
00000000 01000010 01110101 01111010
01111010 00000000
```

Binary machine-code file

Complete hexadecimal view | actual raw binary file attached

```
Offset   Raw bytes (hexadecimal)                          ASCII view

000000   4D 48 43 31 00 40 00 00 00 00 00 B4 10 01 00 00   MHC1.@..........
000010   00 00 00 0A 24 09 00 01 24 0A 00 65 0C 10 00 07   ....$...$..e....
000020   0C 10 00 27 21 29 00 01 15 2A FF FC 08 10 00 2B   ...'!)...*.....+
000030   27 BD FF F8 AF BF 00 04 24 01 00 03 01 21 00 1A   '.......$....!..
000040   00 00 58 10 24 01 00 05 01 21 00 1A 00 00 60 10   ..X.$....!....`.
000050   15 60 00 01 0C 10 00 19 15 80 00 01 0C 10 00 1E   .`..............
000060   11 60 00 02 11 80 00 01 0C 10 00 23 8F BF 00 04   .`.........#....
000070   27 BD 00 08 03 E0 00 08 24 02 00 04 3C 04 10 01   '.......$...<...
000080   34 84 00 00 00 00 00 0C 03 E0 00 08 24 02 00 04   4...........$...
000090   3C 04 10 01 34 84 00 05 00 00 00 0C 03 E0 00 08   <...4...........
0000A0   24 02 00 01 01 20 20 21 00 00 00 0C 03 E0 00 08   $....  !........
0000B0   24 02 00 0B 24 04 00 0A 00 00 00 0C 03 E0 00 08   $...$...........
0000C0   24 02 00 0A 00 00 00 0C 46 69 7A 7A 00 42 75 7A   $.......Fizz.Buz
0000D0   7A 00                                            z.

210 actual bytes; the complete raw file is embedded in this PDF.
Header: offsets 0x00..0x13. Text: 0x14..0xC7. Data: 0xC8..0xD1.
Data bytes: 46 69 7A 7A 00 42 75 7A 7A 00 = Fizz\0Buzz\0.
```

Java test source

Tests.java | listing 1 of 3

```java
import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Arrays;

/** Deterministic encoding, parser, image-format, and rejection tests. */
public final class Tests {
    private static int passed;
    interface Check { void run() throws Exception; }

    static void check(String name, Check check) throws Exception {
        check.run();
        passed++;
        System.out.println("PASS " + name);
    }

    static void require(boolean ok, String message) {
        if (!ok) throw new AssertionError(message);
    }

    static void rejects(String source, String fragment) {
        try {
            Assembler.assemble(source);
            throw new AssertionError("Invalid assembly was accepted");
        } catch (IllegalArgumentException e) {
            require(e.getMessage().contains(fragment), e.getMessage());
            System.out.println("  Expected error: " + e.getMessage());
        }
    }

    static void badImage(Path file, byte[] bytes, String fragment) throws Exception {
        Files.write(file, bytes);
        try {
            Image.read(file);
            throw new AssertionError("Invalid image was accepted");
        } catch (IllegalArgumentException e) {
            require(e.getMessage().contains(fragment), e.getMessage());
            System.out.println("  Expected error: " + e.getMessage());
        }
    }

    public static void main(String[] args) throws Exception {
        check("Part 1 known instruction words", () -> {
            Assembler.Result r = Assembler.assemble(
                "addi $t0, $zero, 3\naddi $t1, $zero, 5\n"
                + "add $t2, $t0, $t1\nsub $t3, $t1, $t0");
            ByteBuffer b = ByteBuffer.wrap(r.text);
            int[] expected = {0x20080003, 0x20090005, 0x01095020, 0x01285822};
            for (int word : expected) require(b.getInt() == word, "Word mismatch");
        });
        check("Forward/backward branch offsets and jump target", () -> {
            byte[] bytes = Assembler.assemble("start: beq $0, $0, end\n"
                + "bne $8, $0, start\nend: j start").text;
            ByteBuffer b = ByteBuffer.wrap(bytes);
            require(b.getInt() == 0x10000001, "Forward branch");
            require(b.getInt() == 0x1500FFFE, "Backward branch");
            require(b.getInt() == 0x08100000, "Jump address");
```

Java test source / continued

Tests.java | listing 2 of 3

```
        });
        check("li widths and immediate boundaries", () -> {
            ByteBuffer b = ByteBuffer.wrap(Assembler.assemble(
                "li $t0, -32768\nli $t0, 65535\nli $t0, 0x12345678").text);
            int[] expected = {0x24088000, 0x3408FFFF, 0x3C081234, 0x35085678};
            for (int word : expected) require(b.getInt() == word, "li encoding");
        });
        check("All supported real instructions and pseudos round trip", () -> {
            String source = ".data\ns: .asciiz \"A#B\\n\"\n.byte 0, 255\n.text\n"
                + "main: li $t0, 0x12345678\nla $a0, s\nmove $t1, $t0\n"
                + "rem $t2, $t1, 3\nnop\naddi $t0, $t0, -1\n"
                + "add $t0, $t1, $t2\nsub $t0, $t1, $t2\n"
                + "sw $ra, -4($sp)\nlw $ra, -4($sp)\n"
                + "beq $t0, $zero, done\nbne $t0, $zero, main\n"
                + "jal done\nj done\ndone: syscall\njr $ra";
            Assembler.Result r = Assembler.assemble(source);
            String rebuilt = Disassembler.disassemble(new Image(r.text, r.data));
            Assembler.Result round = Assembler.assemble(rebuilt);
            require(Arrays.equals(r.text, round.text), "Text mismatch");
            require(Arrays.equals(r.data, round.data), "Data mismatch");
            require(r.data.length == 7 && r.data[1] == '#', "Quoted comment handling");
        });
        check("Parser rejects unsupported instruction with line number", () ->
            rejects(".text\nmul $t0, $t1, $t2", "Line 2: Unsupported instruction"));
        check("Undefined label", () -> rejects("j missing", "Undefined label"));
        check("Duplicate label", () -> rejects("x: nop\nx: nop", "Duplicate label"));
        check("Invalid register", () -> rejects("li $t99, 1", "Unknown register"));
        check("Signed immediate overflow", () ->
            rejects("addi $t0, $0, 32768", "Value out of range"));
        check("Unsigned immediate overflow", () ->
            rejects("ori $t0, $0, -1", "Value out of range"));
        check("Wrong operand count", () -> rejects("add $t0, $t1", "operand(s)"));
        check("Unsupported directive", () -> rejects(".word 1", "Unsupported directive"));
        check("Instruction in data", () -> rejects(".data\nnop", "must be in .text"));
        check("Branch into data", () -> rejects(
            ".data\nx: .asciiz \"x\"\n.text\nj x", "not in .text"));
        check("Unclosed quote", () -> rejects(".data\n.asciiz \"x", "Unclosed quote"));
        check("Zero divisor", () -> rejects("rem $t0, $t1, 0", "cannot be zero"));
        check("Reserved temporary register", () ->
            rejects("rem $t0, $at, 3", "reserves $at"));
        check("Bad byte", () -> rejects(".data\n.byte 256\n.text\nnop", "out of range"));
        check("BOM, CRLF, comment characters, and character literal", () -> {
            Assembler.Result r = Assembler.assemble(
                "\uFEFF.text\r\nli $a0, ',' # comment\r\nnop");
            require(ByteBuffer.wrap(r.text).getInt() == 0x2404002C, "Character literal");
        });
        Path dir = Files.createTempDirectory("mhc-tests-");
        try {
            Path binary = dir.resolve("case.mhc-bin");
            Path ascii = dir.resolve("case.mhc-ascii");
            Assembler.Result r = Assembler.assemble(".data\n.byte 255\n.text\nnop");
            byte[] valid = Image.pack(r.text, r.data);
            check("Both image formats preserve non-word-aligned data", () -> {
                Image.write(dir.resolve("case").toString(), r.text, r.data);
                for (Path file : new Path[]{binary, ascii}) {
                    Image image = Image.read(file);
                    require(Arrays.equals(image.text, r.text), "Text mismatch");
                    require(Arrays.equals(image.data, r.data), "Data mismatch");
```

Java test source / continued

Tests.java | listing 3 of 3

```
                }
                require(Files.size(binary) == 25, "Unexpected padding");
            });
            check("Truncated header", () -> badImage(binary, new byte[4], "Truncated"));
            check("Bad magic", () -> {
                byte[] bad = valid.clone(); bad[0] = 0;
                badImage(binary, bad, "magic");
            });
            check("Wrong starting address", () -> {
                byte[] bad = valid.clone(); bad[4] = 1;
                badImage(binary, bad, "starting addresses");
            });
            check("Wrong segment length", () -> {
                byte[] bad = valid.clone(); bad[11] = 3;
                badImage(binary, bad, "lengths");
            });
            check("Trailing bytes", () ->
                badImage(binary, Arrays.copyOf(valid, valid.length + 1), "lengths"));
            check("Invalid ASCII character", () -> badImage(ascii,
                "010x".getBytes(StandardCharsets.US_ASCII), "only 0, 1"));
            check("Incomplete ASCII byte", () -> badImage(ascii,
                "010".getBytes(StandardCharsets.US_ASCII), "Incomplete"));
            check("Unknown opcode rejected", () -> {
                try {
                    Disassembler.disassemble(new Image(
                        new byte[]{(byte) 0xFC, 0, 0, 0}, new byte[0]));
                    throw new AssertionError("Accepted unknown opcode");
                } catch (IllegalArgumentException e) {
                    require(e.getMessage().contains("Unsupported opcode"), e.getMessage());
                }
            });
            check("Reserved bits rejected instead of silently discarded", () -> {
                try {
                    Disassembler.disassemble(new Image(
                        new byte[]{0, 0x20, 0, 0}, new byte[0]));
                    throw new AssertionError("Accepted sll with nonzero rs");
                } catch (IllegalArgumentException e) {
                    require(e.getMessage().contains("reserved"), e.getMessage());
                }
            });
        } finally {
            Files.deleteIfExists(dir.resolve("case.mhc-bin"));
            Files.deleteIfExists(dir.resolve("case.mhc-ascii"));
            Files.delete(dir);
        }
        System.out.println("ALL " + passed + " TESTS PASSED");
    }
}
```

Verification script

verify.py | listing 1 of 2

```
#!/usr/bin/env python3
"""Run Java tests and round trips; optionally verify execution using MARS 4.5."""
import argparse
import hashlib
from pathlib import Path
import subprocess
import tempfile

ROOT = Path(__file__).resolve().parent


def run(*args):
    result = subprocess.run([str(a) for a in args], cwd=ROOT,
                            text=True, capture_output=True)
    if result.returncode:
        raise RuntimeError(result.stdout + result.stderr)
    return result.stdout


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--mars', type=Path, help='Path to Mars4_5.jar')
    args = parser.parse_args()
    mars = args.mars.resolve() if args.mars else None
    print('Module 5 verification')
    print(subprocess.run(['java', '-version'], text=True, capture_output=True).stderr.strip())
    with tempfile.TemporaryDirectory(prefix='mhc-verify-') as folder:
        temp = Path(folder)
        classes = temp / 'classes'
        classes.mkdir()
        run('javac', '-d', classes, *sorted(ROOT.glob('*.java')))
        java = ['java', '-cp', classes]
        print(run(*java, 'Tests'), end='')
        prefix = temp / 'fizzbuzz'
        message = run(*java, 'Assembler', ROOT / 'fizzbuzz.asm', prefix)
        print(message.replace(str(temp), '<temp>'), end='')
        raw = prefix.with_suffix('.mhc-bin').read_bytes()
        print('Binary image: %d bytes (20 header + 180 text + 10 data)' % len(raw))
        print('SHA-256: ' + hashlib.sha256(raw).hexdigest())
        sources = [ROOT / 'fizzbuzz.asm']
        for kind in ('mhc-ascii', 'mhc-bin'):
            image = prefix.with_suffix('.' + kind)
            checked_in = ROOT / image.name
            assert image.read_bytes() == checked_in.read_bytes(), 'Stale submitted image'
            asm = temp / (kind + '.asm')
            print(run(*java, 'Disassembler', image, asm).replace(str(temp), '<temp>'), end='')
            assert asm.read_bytes() == (ROOT / 'reconstructed.asm').read_bytes()
            rebuilt = temp / (kind + '-rebuilt')
            run(*java, 'Assembler', asm, rebuilt)
            for suffix in ('.mhc-bin', '.mhc-ascii'):
                actual = rebuilt.with_suffix(suffix).read_bytes()
                assert actual == prefix.with_suffix(suffix).read_bytes()
            print('PASS ' + kind + ': reconstructed source and BOTH reassembled files match')
            sources.append(asm)
        # Verify CLI failure status as well as error text, independently of unit tests.
        invalid = temp / 'invalid.asm'
        invalid.write_text('.text\nmul $t0, $t1, $t2\n')
        failure = subprocess.run([str(x) for x in [*java, 'Assembler', invalid]],
```

Verification script / continued

verify.py | listing 2 of 2

```
                                text=True, capture_output=True)
        assert failure.returncode == 1 and 'Line 2' in failure.stderr
        print('PASS CLI invalid input: exit 1; ' + failure.stderr.strip())
        expected = [('Fizz' if n % 3 == 0 else '') + ('Buzz' if n % 5 == 0 else '')
                    or str(n) for n in range(1, 101)]
        if mars:
            for asm in sources:
                # nc: no banner; sm: start at main; no db: delay slots disabled.
                output = run('java', '-jar', mars, 'nc', 'sm', 'ae1', 'se1', '100000', asm)
                assert output.strip().splitlines() == expected, ('FizzBuzz differs: ' + str(asm))
                print('PASS MARS: ' + asm.name + ' prints all 100 expected lines')
            print('Expected output sample: ' + ', '.join(expected[:15]))
        else:
            print('SKIPPED MARS execution: supply --mars /path/to/Mars4_5.jar')
        print('VERIFICATION PASSED')


if __name__ == '__main__':
    main()
```

Captured test evidence

test-evidence.txt | listing 1 of 2

```
Module 5 verification
openjdk version "17.0.20.1" 2026-08-18
OpenJDK Runtime Environment Temurin-17.0.20.1+1 (build 17.0.20.1+1)
OpenJDK 64-Bit Server VM Temurin-17.0.20.1+1 (build 17.0.20.1+1, mixed mode, sharing)
PASS Part 1 known instruction words
PASS Forward/backward branch offsets and jump target
PASS li widths and immediate boundaries
PASS All supported real instructions and pseudos round trip
  Expected error: Line 2: Unsupported instruction: mul
PASS Parser rejects unsupported instruction with line number
  Expected error: Line 1: Undefined label: missing
PASS Undefined label
  Expected error: Line 2: Duplicate label: x
PASS Duplicate label
  Expected error: Line 1: Unknown register: $t99
PASS Invalid register
  Expected error: Line 1: Value out of range: 32768
PASS Signed immediate overflow
  Expected error: Line 1: Value out of range: -1
PASS Unsigned immediate overflow
  Expected error: Line 1: Expected 3 comma-separated operand(s).
PASS Wrong operand count
  Expected error: Line 1: Unsupported directive: .word
PASS Unsupported directive
  Expected error: Line 2: Instructions must be in .text.
PASS Instruction in data
  Expected error: Line 4: Branch/jump target is not in .text: x
PASS Branch into data
  Expected error: Line 2: Unclosed quote.
PASS Unclosed quote
  Expected error: Line 1: rem divisor cannot be zero.
PASS Zero divisor
  Expected error: Line 1: rem reserves $at.
PASS Reserved temporary register
  Expected error: Line 2: Value out of range: 256
PASS Bad byte
PASS BOM, CRLF, comment characters, and character literal
PASS Both image formats preserve non-word-aligned data
  Expected error: Truncated MHC1 header.
PASS Truncated header
  Expected error: Bad MHC1 magic/version.
PASS Bad magic
  Expected error: Unsupported segment starting addresses.
PASS Wrong starting address
  Expected error: Invalid segment lengths or file size.
PASS Wrong segment length
  Expected error: Invalid segment lengths or file size.
PASS Trailing bytes
  Expected error: ASCII image accepts only 0, 1, and whitespace.
PASS Invalid ASCII character
  Expected error: Incomplete byte in ASCII image.
PASS Incomplete ASCII byte
PASS Unknown opcode rejected
PASS Reserved bits rejected instead of silently discarded
ALL 29 TESTS PASSED
Assembled 45 instructions; 10 data bytes.
Wrote <temp>/fizzbuzz.mhc-ascii and <temp>/fizzbuzz.mhc-bin
Binary image: 210 bytes (20 header + 180 text + 10 data)
```

Captured test evidence / continued

test-evidence.txt | listing 2 of 2

```
SHA-256: 521b0b9dbde491f888cdd0c7047bc5a382af40e213d60275bba391153022553f
Wrote <temp>/mhc-ascii.asm; reassembly bytes verified.
PASS mhc-ascii: reconstructed source and BOTH reassembled files match
Wrote <temp>/mhc-bin.asm; reassembly bytes verified.
PASS mhc-bin: reconstructed source and BOTH reassembled files match
PASS CLI invalid input: exit 1; Assembly failed: Line 2: Unsupported instruction: mul
PASS MARS: fizzbuzz.asm prints all 100 expected lines
PASS MARS: mhc-ascii.asm prints all 100 expected lines
PASS MARS: mhc-bin.asm prints all 100 expected lines
Expected output sample: 1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz
VERIFICATION PASSED
```

Captured FizzBuzz output

All 100 output lines from the original program in MARS

```
  1  1
  2  2
  3  Fizz
  4  4
  5  Buzz
  6  Fizz
  7  7
  8  8
  9  Fizz
 10  Buzz
 11  11
 12  Fizz
 13  13
 14  14
 15  FizzBuzz
 16  16
 17  17
 18  Fizz
 19  19
 20  Buzz
 21  Fizz
 22  22
 23  23
 24  Fizz
 25  Buzz
 26  26
 27  Fizz
 28  28
 29  29
 30  FizzBuzz
 31  31
 32  32
 33  Fizz
 34  34
 35  Buzz
 36  Fizz
 37  37
 38  38
 39  Fizz
 40  Buzz
 41  41
 42  Fizz
 43  43
 44  44
 45  FizzBuzz
 46  46
 47  47
 48  Fizz
 49  49
 50  Buzz
 51  Fizz
 52  52
 53  53
 54  Fizz
 55  Buzz
 56  56
 57  Fizz
 58  58
 59  59
 60  FizzBuzz
 61  61
 62  62
 63  Fizz
 64  64
 65  Buzz
 66  Fizz
 67  67
 68  68
 69  Fizz
 70  Buzz
 71  71
 72  Fizz
 73  73
 74  74
 75  FizzBuzz
 76  76
 77  77
 78  Fizz
 79  79
 80  Buzz
 81  Fizz
 82  82
 83  83
 84  Fizz
 85  Buzz
 86  86
 87  Fizz
 88  88
 89  89
 90  FizzBuzz
 91  91
 92  92
 93  Fizz
 94  94
 95  Buzz
 96  Fizz
 97  97
 98  98
 99  Fizz
100  Buzz
```